PoisonShield — Technical Documentation

1. Project Overview

PoisonShield is a multimodal data-auditing framework for detecting and mitigating data poisoning in tabular and image datasets. It is built as a Streamlit application with a modular backend. It does not train a classifier on uploaded data as its primary purpose; instead it audits data quality, feature relationships, label consistency, duplicates, anomalies and dataset structure.

Core principles:

- All dashboard metrics come from real analysis.
- No simulated poisoning in the normal workflow.
- No hardcoded results.
- Original uploaded files are never modified.
- Sanitisation writes new copies only.
- Detection results are review priorities, not proof of poisoning.

2. High-Level Architecture

PoisonShield has four layers:

1. **Ingestion layer**
- Reads tabular files and image archives.
- Profiles datasets.
- Preserves original files.

2. **Feature layer**
- Converts tabular data into numerical feature matrices.
- Extracts image embeddings.
- Generates out-of-sample predicted probabilities.

3. **Auditing and intelligence layer**
- Runs Cleanlab Datalab.
- Builds sample relationships.
- Computes unified risk.
- Generates explanations.

4. **Presentation and export layer**

- Streamlit UI with 11 sections.
- Human review.
- Sanitisation.
- Reports and exports.

3. End-to-End Flow

1. User uploads a dataset.
2. PoisonShield copies the original file unchanged.
3. The dataset is profiled.
4. Features are built.
5. Out-of-sample probabilities are computed.
6. Cleanlab Datalab runs applicable detectors.
7. Relationship analysis builds KNN neighbourhoods.
8. Risk engine fuses all evidence into per-sample scores.
9. Explanations are generated for flagged samples.
10. Artifacts are saved.
11. User reviews suspicious samples.
12. User generates a cleaned copy if desired.
13. User exports reports.

4. Data Ingestion

4.1 Tabular Ingestion

Supported formats: CSV, XLSX, XLS.

Processing:

- Read file with encoding fallbacks for CSV.
- Apply row and column limits.
- Detect missing values.
- Detect duplicate rows using exact full-row equality.
- Infer numerical, categorical, datetime, boolean columns.
- Detect identifier-like, constant, all-missing, high-cardinality columns.
- Detect invalid numeric values and missing sentinels.
- Preserve the original dataframe.

Label handling:

- Label column is optional.
- If no label column exists, the dataset is treated as unlabelled.
- Label-aware detectors are skipped for unlabelled data.

Ground truth:

- A separate evaluation column can be declared.
- It is excluded from features to avoid leakage.

4.2 Image Ingestion

Supported inputs:

- ZIP archive
- Batch of image files
- Folder structure with class names
- Optional label mapping CSV/XLSX

Processing:

- Safe ZIP extraction with path-traversal protection.
- Image validation and decoding.
- Pixel safety limit to prevent decompression bombs.
- SHA-256 hashing for exact duplicates.
- Brightness and entropy for low-information detection.
- Format mismatch detection.
- Unreadable file retention as integrity findings.

Label mapping:

- Path and label columns are auto-detected.
- Mapping can be provided explicitly.
- Folder names can be used as labels.
- Extra mapping columns are available for evaluation but never used as image features.

5. Feature Engineering

5.1 Tabular Features

The tabular feature builder:

- Separates numerical and categorical columns.
- Coerces numeric-like text to numbers.
- Normalises categorical values.
- Handles datetime columns.
- Applies missing-value imputation.
- One-hot encodes categorical variables.
- Scales numerical features.
- Optionally applies SVD for dimensionality reduction.
- Clips extreme values.
- Excludes configured columns such as ground truth.

Output:

- A dense numerical matrix aligned with sample IDs.
- Metadata describing preprocessing.

5.2 Image Features

Supported extractors:

1. DINOv2 ViT-S/14
2. DINOv2 ViT-B/14
3. ResNet-50
4. Offline handcrafted descriptor

Extraction process:

- Load image.
- Convert to RGB.
- Resize according to extractor.
- Apply extractor-specific transform.
- Run model in inference mode.
- Normalise embeddings.
- Cache embeddings by SHA-256.
- Process in batches.

Fallback behaviour:

- If DINOv2 cannot load, use ResNet-50.
- If ResNet-50 cannot load, use handcrafted descriptor.

- The actual extractor used is recorded in metadata.

Handcrafted descriptor:

- Colour histograms.
- Normalised 16x16 greyscale patch.
- Gradient magnitude and orientation histogram.
- Mean and standard deviation.
- Fixed dimension: 324.

6. Predicted Probabilities

For labelled datasets, PoisonShield generates out-of-sample class probabilities using cross-fitting.

Process:

- Split data into folds.
- Train a logistic regression model on training folds.
- Predict probabilities on the held-out fold.
- Combine predictions across folds.
- Align probability columns to Cleanlab's class order.

If logistic regression fails, KNN-based probabilities are used as a fallback.

These probabilities are model-based evidence, not ground truth.

7. Cleanlab Datalab Integration

PoisonShield uses Cleanlab Datalab as the real detector layer.

Detectors:

- Label issues
- Outliers
- Near duplicates
- Class imbalance
- Non-IID

Implementation details:

- Datalab is initialised with the dataset, task, and label column.
- Feature matrices and predicted probabilities are passed where supported.

- Detector statuses are recorded.
- Failed detectors are reported explicitly.
- Dataset-level findings are separated from per-sample findings.

Important behaviour:

- Cleanlab scores are quality scores: higher is better.
- PoisonShield converts them to risk evidence as `1 - quality`.
- Class imbalance and non-IID are dataset-level findings and are excluded from per-sample risk.

8. Relationship Engine

The relationship engine builds sample neighbourhoods using KNN.

For tabular data:

- Uses the feature matrix.
- Supports Euclidean and cosine distance.

For image data:

- Uses image embeddings.
- Uses cosine distance.

Outputs:

- Nearest neighbours.
- Pairwise distances.
- Similarities.
- Labelled neighbour counts.
- Neighbour label agreement.
- Majority neighbour label.
- Conflicting neighbourhood flag.
- Mean distance.
- Mean similarity.
- Isolation percentile.
- Top neighbour and top similarity.
- Suggested label and confidence.

Projection:

- PCA is used for 2-D projection.

- DBSCAN is used for visual clusters.
- Explained variance is reported.

Relationship analysis is supporting evidence, not proof of poisoning.

9. Risk Engine

The risk engine combines multiple evidence sources into a unified per-sample risk score.

Evidence groups:

- Label and neighbourhood: 0.50
- Outlier: 0.20
- Duplicate: 0.10
- Integrity: 0.20

Label and neighbourhood signals are fused non-additively because they are correlated.

Risk formula:

For each sample:

1. Compute each group signal as `1 - quality` where applicable.
2. Multiply by group weight.
3. Sum weighted signals.
4. Divide by the sum of available group weights.
5. Clip to [0, 1].

Unavailable groups are excluded and coverage is reported.

Risk categories:

- Low: score <= 0.30
- Moderate: score <= 0.55
- High: score <= 0.75
- Critical: score > 0.75

Exact duplicates:

- Tabular: rows identical across all columns.
- Image: byte-identical files by SHA-256.
- Every member of a duplicate group is marked.

10. Explainability Engine

For every flagged sample, PoisonShield generates a structured explanation.

Findings include:

- Label quality
- Neighbourhood consistency
- Similarity
- Outlier status
- Duplicate status
- Integrity issues

Each finding has:

- Kind
- Severity
- Text
- Evidence

The narrative summary is built from actual detector outputs and calculated evidence. No fabricated explanations are generated.

11. Human Review and Sanitisation

Review actions:

- Approve suggested correction
- Reject suggested correction
- Retain original label
- Remove sample
- Mark as uncertain
- Clear decision

Each decision stores:

- Sample ID
- Action
- Original label
- Suggested label

- Corrected label
- Notes
- Reviewer
- Timestamp

Sanitisation:

- Applies decisions to a new copy.
- Never modifies the original.
- Tabular output can be CSV or XLSX.
- Image output is a ZIP archive.
- Changes are recorded in a changes table.

12. User Interface Sections

1. Overview

- Dataset health
- Risk distribution
- Detector completion
- Warnings
- Processing time and memory
- Provenance status

2. Upload

- Tabular and image upload
- Label and ground-truth configuration
- Analysis settings
- Risk weights and thresholds

3. Inspection

- Dataset profiling
- Missing values
- Duplicates
- Image gallery and integrity

4. Tabular Analysis

- Detector status

- Cleanlab summary
- Risk signals
- Flagged samples
- Feature representation

5. Image Analysis

- Extractor metadata
- Risk signals
- Image gallery
- Integrity and duplicates

6. Relationship Explorer

- PCA projection
- Neighbour edges
- Cluster visualisation
- Sample inspection

7. Suspicious Samples

- Filtering
- Triage table
- Inspector
- Flag overview

8. Label Review

- Review queue
- Decision recording
- History
- Decision summary

9. Sanitisation

- Decision summary
- Cleaned copy generation
- Downloads
- Provenance verification

10. Model / Quality Evaluation

- Detection metrics
- Model comparison
- Quality summary
- Limitations

11. Reports / Export

- Report archive
- CSV exports
- Session metadata
- Artifact listing

13. Evaluation

Detection metrics:

- Precision
- Recall
- F1-score
- Accuracy
- ROC AUC
- Average precision

Metrics are computed against a real ground-truth column.

Model comparison:

- Trains the same model on original vs cleaned labels
- Evaluates on a separate untouched dataset
- Reports delta
- Does not claim improvement

14. Configuration and Limits

Processing limits:

- Max tabular rows: 200,000
- Max tabular columns: 1,000
- Max images: 3,000
- Max upload size: 1,024 MB

- Max embedding batch: 32
- Image pixel limit: 80,000,000

15. File and Module Map

Backend:

- `backend/pipeline.py`: orchestration
- `backend/auditing/cleanlab_engine.py`: Cleanlab integration
- `backend/intelligence/relationship_engine.py`: relationships
- `backend/intelligence/risk_engine.py`: risk scoring
- `backend/intelligence/explainability.py`: explanations
- `backend/intelligence/reviewer.py`: review decisions
- `backend/intelligence/sanitizer.py`: sanitisation
- `backend/features/tabular_features.py`: tabular features
- `backend/features/image_features.py`: image embeddings
- `backend/ingestion/tabular.py`: tabular ingestion
- `backend/ingestion/images.py`: image ingestion
- `backend/evaluation/metrics.py`: detection metrics
- `backend/evaluation/model_comparison.py`: model comparison
- `backend/reporting/artifacts.py`: artifact persistence
- `backend/reporting/exporter.py`: reports and exports

UI:

- `ui/app.py`: shell
- `ui/state.py`: session state
- `ui/components.py`: reusable components
- `ui/sections/*.py`: 11 pages
- `ui/theme.py`: styling

16. Testing

Tests cover:

- Tabular labelled and unlabelled ingestion
- Image ingestion and mapping
- Tabular and image pipelines
- Exact duplicates

- Sanitisation
- Evaluation metrics
- Model comparison

Command:

```
pytest -q
```

17. Limitations

- Risk scores are review priorities, not proof of poisoning.
- Neighbourhood analysis can be misled by correlated errors.
- Image embeddings depend on extractor and domain.
- Detection metrics depend on ground-truth quality.
- Duplicate files are not automatically poisoning.
- Model comparison is one model family on one split.
- Sanitisation does not guarantee improved accuracy.
- Streamlit Community Cloud may throttle heavy workloads.

18. Summary

PoisonShield combines Cleanlab Datalab with custom relationship, risk, explainability, review, sanitisation, evaluation and reporting layers to provide a complete multimodal data-auditing platform. It is designed to be explainable, modular, and suitable for both research and practical data-quality workflows.